COER UNIVERSITY

ROORKEE, UTTARAKHAND

LAB PROJECT REPORT

SMART DIGITAL EXPENSE TRACKER WITH BUDGET PREDICTION

BCA I Semester | Session 2026–2027

Submitted by	Harshit Tyagi
Submitted to	Mr. Naman Chauhan
Course	Bachelor of Computer Applications (BCA)
Subject	C++ Programming Lab
University	COER University, Roorkee

Academic Year: 2026–2027

1. Introduction

The Smart Digital Expense Tracker is a menu-driven C++ application designed to help users record and manage their daily expenses. It stores expense details such as date, category, amount and description. The program also provides monthly summaries, budget alerts and a simple estimate of future monthly spending.

2. Objective

To develop a simple expense management system that helps users record daily expenses, categorize spending, monitor a monthly budget, search or delete records, and predict whether the monthly budget may be exceeded.

3. Problem Statement

Many students spend money every day but do not maintain a proper record. As a result, they may lose track of where their money is going and may exceed their monthly budget. This project provides a basic digital method to save expenses, review spending and receive an alert when spending approaches or exceeds the budget.

4. Requirements

Hardware: Computer or laptop.
Software: C++ compiler (such as GCC/MinGW), Code::Blocks/VS Code/Visual Studio, and a text editor.

5. Modules

Module	Description
User Registration and Login	Allows a user to create an account or log in using a username and password.
Add Daily Expense	Stores date, category, amount and description for a new expense.
View Expense History	Displays all saved expense records.
Category-wise Expense	Calculates the total spending for each category.
Monthly Expense Report	Displays expenses for a selected month and calculates the total.
Budget Setting and Warning	Stores a monthly budget and displays a warning at 80% usage or when the budget is exceeded.
Expense Prediction	Estimates 30-day spending using the average daily spending recorded so far.
Search Expense	Finds a record using its unique expense ID.
Delete Expense	Deletes a selected record using its ID.
File Saving	Stores records in expenses.txt and the budget in budget.txt so they remain available after restarting the prog

6. Concepts Used

Variables and data types, structures, vectors, functions, loops, conditional statements, file handling, string handling, searching, sorting-related library utilities, input/output streams and menu-driven programming.

7. Algorithm / Logic

• Start the program and load saved expenses and budget from files.

• Ask the user to register or log in.

• If authentication is successful, display the main menu.

• Accept the user's menu choice.

• Perform the selected operation: add, view, summarize, report, set budget, predict, search or delete.

• After changes, save updated records to files.

• Repeat the menu until the user selects Save and Exit.

• Stop the program.

8. Budget Prediction Formula

The program calculates the total spending recorded for the selected month and finds the latest day number with a recorded expense. Estimated 30-day spending = (Total spending recorded ÷ Latest recorded day number) × 30. This is a simple estimate based on the assumption that spending continues at the same average daily rate; it is not an AI prediction.

9. Source Code (C++)

Save the following program as **expense_tracker.cpp**. Compile using a C++11-or-later compiler. The program creates its data files in the current working folder.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <iomanip>
#include <string>
#include <algorithm>
#include <limits>
using namespace std;

struct Expense {
    int id;
    string date;       // YYYY-MM-DD
    string category;
    string description;
    double amount;
};

vector<Expense> expenses;
double monthlyBudget = 0.0;
string currentUser;

void loadData() {
    expenses.clear();
    ifstream in("expenses.txt");
    Expense e;
    while (in >> e.id >> e.date >> e.category >> e.amount) {
        in.ignore(numeric_limits<streamsize>::max(), '|');
        getline(in, e.description);
        expenses.push_back(e);
    }
    in.close();

    ifstream bin("budget.txt");
    if (bin) bin >> monthlyBudget;
}

void saveData() {
    ofstream out("expenses.txt");
    for (const auto &e : expenses)
        out << e.id << ' ' << e.date << ' ' << e.category << ' '
            << fixed << setprecision(2) << e.amount << '|' << e.description << '\n';
    out.close();

    ofstream bout("budget.txt");
    bout << fixed << setprecision(2) << monthlyBudget;
}

bool registerUser() {
    string username, password, u, p;
    cout << "Create username: ";
    cin >> username;
    cout << "Create password: ";
    cin >> password;

    ifstream in("users.txt");
    while (in >> u >> p) {
        if (u == username) {
            cout << "Username already exists.\\n";
            return false;
        }
    }
    in.close();

    ofstream out("users.txt", ios::app);
    out << username << ' ' << password << '\\n';
    currentUser = username;
    cout << "Registration successful.\\n";
    return true;
}

bool loginUser() {
    string username, password, u, p;
    cout << "Username: ";
```

```
    cin >> username;
    cout << "Password: ";
    cin >> password;

    ifstream in("users.txt");
    while (in >> u >> p) {
        if (u == username && p == password) {
            currentUser = username;
            cout << "Login successful.\\n";
            return true;
        }
    }
    cout << "Invalid username or password.\\n";
    return false;
}

int nextId() {
    int id = 1;
    for (const auto &e : expenses) id = max(id, e.id + 1);
    return id;
}

void addExpense() {
    Expense e;
    e.id = nextId();
    cout << "Date (YYYY-MM-DD): ";
    cin >> e.date;
    cout << "Category (Food/Travel/Study/Shopping/Other): ";
    cin >> e.category;
    cout << "Amount (Rs.): ";
    cin >> e.amount;
    cin.ignore(numeric_limits<streamsize>::max(), '\\n');
    cout << "Description: ";
    getline(cin, e.description);

    if (e.amount <= 0) {
        cout << "Amount must be greater than zero.\\n";
        return;
    }
    expenses.push_back(e);
    saveData();
    cout << "Expense saved. ID = " << e.id << '\\n';
    if (monthlyBudget > 0) {
        double total = 0;
        for (const auto &x : expenses)
            if (x.date.substr(0, 7) == e.date.substr(0, 7)) total += x.amount;
        if (total > monthlyBudget)
            cout << "WARNING: Monthly budget exceeded!\\n";
        else if (total >= monthlyBudget * 0.80)
            cout << "Alert: You have used 80% or more of your budget.\\n";
    }
}

void showExpenses(const vector<Expense> &items) {
    if (items.empty()) {
        cout << "No expenses found.\\n";
        return;
    }
    cout << left << setw(6) << "ID" << setw(13) << "Date"
         << setw(14) << "Category" << setw(12) << "Amount" << "Description\\n";
    cout << string(65, '-') << '\\n';
    for (const auto &e : items)
        cout << left << setw(6) << e.id << setw(13) << e.date
             << setw(14) << e.category << setw(12) << fixed << setprecision(2)
             << e.amount << e.description << '\\n';
}

void viewHistory() {
    showExpenses(expenses);
}

void categorySummary() {
    if (expenses.empty()) { cout << "No expenses available.\\n"; return; }
    vector<pair<string, double>> totals;
    for (const auto &e : expenses) {
        auto it = find_if(totals.begin(), totals.end(),
            [&](const auto &p) { return p.first == e.category; });
        if (it == totals.end()) totals.push_back({e.category, e.amount});
        else it->second += e.amount;
    }
    cout << "\\nCategory-wise spending:\\n";
    for (const auto &p : totals)
        cout << p.first << " : Rs. " << fixed << setprecision(2) << p.second << '\\n';
}

void monthlyReport() {
    string month;
    cout << "Enter month (YYYY-MM): ";
    cin >> month;
    double total = 0;
    cout << "\\nExpenses for " << month << ":\\n";

    for (const auto &e : expenses) {
        if (e.date.substr(0, 7) == month) {
            showExpenses(vector<Expense>{e});
            total += e.amount;
        }
    }
    cout << "Total spending: Rs. " << fixed << setprecision(2) << total << '\\n';
    if (monthlyBudget > 0)
        cout << "Budget: Rs. " << monthlyBudget
             << " | Remaining: Rs. " << monthlyBudget - total << '\\n';
}
```

```
void setBudget() {
    cout << "Enter monthly budget (Rs.): ";
    cin >> monthlyBudget;
    if (monthlyBudget <= 0) {
        cout << "Budget must be greater than zero.\\n";
        monthlyBudget = 0;
        return;
    }
    saveData();
    cout << "Monthly budget saved.\\n";
}

void predictSpending() {
    string month;
    cout << "Enter month (YYYY-MM): ";
    cin >> month;
    double total = 0;
    int days = 0;
    for (const auto &e : expenses) {
        if (e.date.substr(0, 7) == month) {
            total += e.amount;
            days = max(days, stoi(e.date.substr(8, 2)));
        }
    }
    if (days == 0) { cout << "No expenses found for this month.\\n"; return; }
    double prediction = (total / days) * 30.0;
    cout << "Spending recorded so far: Rs. " << fixed << setprecision(2) << total << '\\n';
    cout << "Estimated 30-day spending: Rs. " << prediction << '\\n';
    if (monthlyBudget > 0 && prediction > monthlyBudget)
        cout << "Prediction: Monthly budget may be exceeded.\\n";
    else
        cout << "Prediction: Spending is within the set budget.\\n";
}

void searchExpense() {
    int id;
    cout << "Enter expense ID: ";
    cin >> id;
    for (const auto &e : expenses) {
        if (e.id == id) { showExpenses(vector<Expense>{e}); return; }
    }
    cout << "Expense not found.\\n";
}
void deleteExpense() {
    int id;
    cout << "Enter expense ID to delete: ";
    cin >> id;
    auto oldSize = expenses.size();
    expenses.erase(remove_if(expenses.begin(), expenses.end(),
        [&](const Expense &e) { return e.id == id; }), expenses.end());
    if (expenses.size() == oldSize) cout << "Expense ID not found.\\n";
    else { saveData(); cout << "Expense deleted.\\n"; }
}

int main() {
    loadData();
    int choice, auth;
    cout << "===== SMART DIGITAL EXPENSE TRACKER =====\\n";
    cout << "1. Register\\n2. Login\\nChoose: ";
    cin >> auth;
    bool loggedIn = (auth == 1) ? registerUser() : (auth == 2 ? loginUser() : false);
    if (!loggedIn) {
        cout << "Authentication failed. Exiting.\\n";
        return 0;
    }

    do {
        cout << "\\n===== MAIN MENU =====\\n"
             << "1. Add Daily Expense\\n2. View Expense History\\n"
             << "3. Category-wise Summary\\n4. Monthly Expense Report\\n"
             << "5. Set Monthly Budget\\n6. Predict Monthly Spending\\n"
             << "7. Search Expense by ID\\n8. Delete Expense\\n"
             << "9. Save and Exit\\nEnter choice: ";
        cin >> choice;
        switch (choice) {
            case 1: addExpense(); break;
            case 2: viewHistory(); break;
            case 3: categorySummary(); break;
            case 4: monthlyReport(); break;
            case 5: setBudget(); break;
            case 6: predictSpending(); break;
            case 7: searchExpense(); break;
            case 8: deleteExpense(); break;
            case 9: saveData(); cout << "Data saved. Thank you!\\n"; break;
            default: cout << "Invalid choice. Try again.\\n";
        }
    } while (choice != 9);
    return 0;
}
```

10. Sample Input and Output

```
===== SMART DIGITAL EXPENSE TRACKER =====
1. Register
2. Login
Choose: 1
Create username: harshit
Create password: demo123
Registration successful.
===== MAIN MENU =====
```

```
1. Add Daily Expense
2. View Expense History
...
5. Set Monthly Budget
Enter choice: 5
Enter monthly budget (Rs.): 5000
Monthly budget saved.

Enter choice: 1
Date (YYYY-MM-DD): 2026-11-12
Category (Food/Travel/Study/Shopping/Other): Food
Amount (Rs.): 150
Description: Lunch
Expense saved. ID = 1

Enter choice: 9
Data saved. Thank you!
```

The sample is illustrative. Actual totals and prediction messages depend on the records entered by the user.

11. How to Compile and Run

1. Save the source code as **expense_tracker.cpp**.
2. Open a terminal in the folder containing the file.
3. Compile: **g++ expense_tracker.cpp -o expense_tracker**
4. Run on Windows: **expense_tracker.exe**; on macOS/Linux: **./expense_tracker**.

12. Conclusion

The Smart Digital Expense Tracker demonstrates how C++ programming concepts can be used to solve a practical day-to-day problem. It provides basic expense recording, category summaries, monthly reporting, budget alerts, search, deletion and file storage. The prediction feature gives a simple estimate to help users plan their spending.

13. Future Enhancements

The project can be improved by adding stronger password protection, separate expense files for multiple users, graphical user interface, charts, export to PDF/Excel, mobile application support, cloud backup, QR bill scanning and more advanced spending prediction.

14. Learning Outcomes

Through this project, students practise menu-driven programming, structures, vectors, functions, loops, conditional statements, file handling, searching, data management, report generation and basic problem-solving.

15. References

1. BCA111P Lab Sheet-09: Smart Digital Expense Tracker with Budget Prediction, COER University.
2. C++ standard library documentation and classroom notes.

Note: This is an academic demonstration project. The sample login stores passwords in a plain-text local file; do not use real or sensitive passwords. For a production system, password hashing and separate user data storage should be implemented.